Reporting

Pytest HTML Test Report in Playwright (using pytest-html)

Overview

pytest-html is a plugin that helps generate **beautiful HTML reports** for your Playwright tests. These reports show which tests passed, failed, or were skipped — and you can even attach screenshots for failed tests.

Official documentation: https://pypi.org/project/pytest-html/

Step 1: Install the plugin

Run the following command in your terminal or command prompt:

```
pip install pytest-html
```

Step 2: Configure pytest.ini

Add these lines to your **pytest.ini** file (in the project root folder):

```
[pytest]
addopts = --headed
    --browser=chromium
    --video=retain-on-failure
    --screenshot=only-on-failure
    --tracing=retain-on-failure
    --html=myreport.html --self-contained-html --capture=tee-sys
```

Explanation:

- --headed → Runs the browser in UI mode.
- --browser=chromium → Specifies the browser to use.
- --video, --screenshot, --tracing → Captures test evidence on failure.
- --html=myreport.html → Generates an HTML report file.
- --self-contained-html → Embeds CSS and JS into one file (no external dependencies).
- --capture=tee-sys → Displays logs in both console and report.

Step 3: Attach Screenshots on Test Failures

Add the following code to your **conftest.py** file.
It captures a screenshot whenever a test fails and attaches it to the HTML report.

```
from pathlib import Path

import pytest

from slugify import slugify


@pytest.hookimpl(hookwrapper=True)

def pytest_runtest_makereport(item, call):

    pytest_html = item.config.pluginmanager.getplugin("html")

    outcome = yield

    screen_file = ''

    report = outcome.get_result()

    extra = getattr(report, "extra", [])


    if report.when == "call":

        xfail = hasattr(report, "wasxfail")

        if report.failed or xfail and "page" in item.funcargs:

            page = item.funcargs["page"]

            screenshot_dir = Path("screenshots")

            screenshot_dir.mkdir(exist_ok=True)

            screen_file = str(screenshot_dir / f"{slugify(item.nodeid)}.png")

            page.screenshot(path=screen_file)


        if (report.skipped and xfail) or (report.failed and not xfail):

            extra.append(pytest_html.extras.png(screen_file))

        report.extra = extra
```

Tip:

- The screenshot will be saved inside a folder named **screenshots**.
- The image will also appear in the HTML report for easy viewing.

Step 4: Create Sample Tests

Example test file: **test_sample.py**

```
from playwright.sync_api import Page, expect
def test_google_search(page):
    page.goto("https://www.google.com")
    expect(page).to_have_title("Google")

def test_bing_search(page):
    page.goto("https://www.bing.com/")
    expect(page).to_have_title("Bing123")  # Intentionally failed
```

Step 5: Run the Tests and Generate Report

Run your tests using:

```
pytest
```

This will generate a file named **myreport.html** in your project folder.

Optional: Add a Timestamp to the Report Name

You can add the current date and time to your report name when running tests via command line.

For mac/Linux:

```
pytest test_sample.py --html=reports/report_$(date +"%Y-%m-%d_%H-%M-%S").html --self-contained-html
```

For Windows (PowerShell):

```
pytest test_sample.py --html="report_$(Get-Date -Format 'yyyy-MM-dd_HH-mm-ss').html" --self-contained-html
```

Allure Test Report (using pytest) in Playwright

Overview

Allure Report is a powerful and visually appealing reporting tool for test automation frameworks like **Pytest**.
It shows detailed information such as test steps, logs, screenshots, and execution history in an interactive dashboard.

Step 1: Install Allure Pytest Plugin

Run the following command to install the required plugin:

```
pip install allure-pytest
```

Step 2: Install Allure Command-Line Tool

The **Allure CLI** (Command-Line Interface) is needed to view and generate reports.

For Windows:

1. Go to https://github.com/allure-framework/allure2/releases
2. Download the latest .zip file.
3. Extract it and **add the bin folder** path to your **system PATH environment variable**.

For macOS:

```
brew install allure
```

For Linux:

```
sudo snap install allure --classic
```

Step 3: Configure Allure in pytest.ini

Add the following line to your pytest.ini file:

```
[pytest]
addopts = --alluredir=reports/allure-results
```

Explanation:

- **--alluredir** specifies the folder where test execution results will be stored (in JSON format).
- Later, these results will be converted into an HTML report.

Step 4: Attach Screenshots on Test Failures (conftest.py)

```
from pathlib import Path
import pytest
from slugify import slugify
import allure

@pytest.hookimpl(hookwrapper=True)
def pytest_runtest_makereport(item, call):
    pytest_html = item.config.pluginmanager.getplugin("html")
    outcome = yield
    screen_file = ''
    report = outcome.get_result()
    extra = getattr(report, "extra", [])
    if report.when == "call":
        xfail = hasattr(report, "wasxfail")
        if report.failed or xfail and "page" in item.funcargs:
            page = item.funcargs["page"]
            screenshot_dir = Path("screenshots")
            screenshot_dir.mkdir(exist_ok=True)
            screen_file = str(screenshot_dir / f"{slugify(item.nodeid)}.png")
            page.screenshot(path=screen_file)

        if (report.skipped and xfail) or (report.failed and not xfail):
            # add the screenshots to the html report
            extra.append(pytest_html.extras.png(screen_file))

            # Attach to Allure report
            allure.attach.file(
                screen_file,
                name="Failure Screenshot",
                attachment_type=allure.attachment_type.PNG
            )
        report.extra = extra
```

Step 5: Create and Run Your Tests

Write a few Playwright tests as usual.

Example:

```
from playwright.sync_api import Page, expect

def test_google_search(page):

    page.goto("https://www.google.com")

    expect(page).to_have_title("Google")
```

```
def test_bing_search(page):
    page.goto("https://www.bing.com")
    expect(page).to_have_title("Bing123")  # Intentionally failed
```

Run your tests using:

```
pytest
```

After the test run, Allure will create a folder:

```
reports/allure-results
```

Step 6: Generate and View the Allure Report

There are two ways to view the report:

Option 1: Temporary report (auto-opens in browser)

```
allure serve reports/allure-results
```

This will **generate and launch** the report in your browser automatically.
(Report will not be saved permanently.)

Option 2: Permanent report (saved for future reference)

```
allure generate reports/allure-results -o reports/allure-report --clean
```

Then open it manually:

```
allure open reports/allure-report
```

Combine Reports (e.g., HTML + Allure)

You can combine multiple report formats in one run:

```
pytest --html=/report.html --junitxml=reports/results.xml --alluredir=reports/allure-results
```

HTML Report

Step 1: Install the plugin

Step 2: Configure pytest.ini

Step 3: Attach Screenshots on Test Failures (conftest.py)

Step 4: Create Sample Tests

Step 5: Run the Tests and Generate Report

Allure report

Step 1: Install Allure Pytest Plugin

Step 2: Install Allure Command-Line Tool (to view and generate reports)

Step 3: Configure Allure in pytest.ini

Step 4: Attach Screenshots on Test Failures (conftest.py)

Step 5: Create and Run Your Tests

Step 6: Generate and View the Allure Report